



Московский государственный университет имени М.В. Ломоносова

Факультет вычислительной математики и кибернетики

Кафедра суперкомпьютеров и квантовой информатики

Отчет по заданию практикума

Генетические алгоритмы

Группа: 523

Повжик Юрий Максимович
Почта: s02210514@gse.cs.msu.ru

Москва, 2025

Содержание

1	Программный код.....	3
2	Графики	8

1 Программный код

Для запуска программы необходимо указать 7 аргументов:

- Число генов
- Размер популяции, кратный числу процессов
- Число итераций
- Вероятность мутации
- Амплитуду изменения гена при мутации
- Частоту миграций
- Целевую функцию: сферическая функция - 0, функция Розенброка - 1 и функция Растригина - 2

Код программы:

```
#include <iostream>
#include <cmath>
#include <mpi.h>

using namespace std;

double mutationP;
double mutationLen;
int migrationInterval;
int evalFunction;

double frand() {
    return double(rand())/RAND_MAX;
}

double sphereF(double* x, int n) {
    double sum = 0.0;
    for (int i = 0; i < n; ++i)
        sum += x[i] * x[i];
    return sum;
}

double rosenbrockF(double* x, int n) {
    double sum = 0.0;
    for (int i = 0; i < n-1; ++i) {
        double t1 = x[i+1] - x[i]*x[i];
        double t2 = x[i] - 1.0;
        sum += 100.0 * t1*t1 + t2*t2;
    }
    return sum;
}

double rastriginF(double* x, int n) {
    double sum = 10.0 * n;
    for (int i = 0; i < n; ++i)
        sum += x[i]*x[i] - 10.0 * cos(2.0 * 3.14 * x[i]);
    return sum;
}
```

```

}

double eval(double* x, int n) {
    switch(evalFunction) {
        case 0: return sphereF(x, n);
        case 1: return rosenbrockF(x, n);
        case 2: return rastriginF(x, n);
        default: return sphereF(x, n);
    }
}

void init(double* P, int m, int n, double xmin=-100, double xmax=100) {
    for(int k=0; k<m; k++)
        for(int i=0; i<n; i++)
            P[k*n+i] = xmin + (xmax - xmin)*frand();
}

void shuffle(double* P, int m, int n) {
    for(int k=0; k<m; k++) {
        int l = rand()%m;
        for(int i=0; i<n; i++)
            swap(P[k*n+i], P[l*n+i]);
    }
}

void select(double* P, int m, int n) {
    double pwin = 0.9;
    shuffle(P, m, n);
    for (int k = 0; k < m / 2; k++) {
        int a = 2 * k;
        int b = 2 * k + 1;
        double fa = eval(P + a * n, n);
        double fb = eval(P + b * n, n);
        double p = frand();
        if ((fa < fb && p < pwin) || (fa > fb && p > pwin))
            for (int i = 0; i < n; i++)
                P[b * n + i] = P[a * n + i];
        else
            for (int i = 0; i < n; i++)
                P[a * n + i] = P[b * n + i];
    }
}

void crossover(double* P, int m, int n) {
    shuffle(P, m, n);
    for( int k=0; k<m/2; k++ )
    {
        int a = 2*k;
        int b = 2*k+1;
        int j = rand()%n;

```

```

        for( int i=j; i<n; i++ )
            swap(P[a*n+i],P[b*n+i]);
    }
}

void mutate(double* P, int m, int n) {
    for (int k = 0; k < m; k++) {
        for (int i = 0; i < n; i++) {
            if (frand() < mutationP) {
                P[k * n + i] += (frand() * 2.0 * mutationLen) - mutationLen;
                P[k * n + i] = std::max(-100.0, std::min(100.0, P[k * n + i]));
            }
        }
    }
}

double printthebest(double* P, int m, int n) {
    int k0 = -1;
    double f0 = 1e12;
    for (int k = 0; k < m; k++) {
        double f = eval(P + k * n, n);
        if (f < f0) {
            f0 = f;
            k0 = k;
        }
    }
    // cout << f0 << ": ";
    // for (int i = 0; i < n; i++)
    //     cout << P[k0 * n + i];
    // cout << endl;
    return f0;
}

void migrate(double* P, int m, int n, int rank, int size) {
    MPI_Status status;
    int next = (rank + 1) % size;
    int prev = (rank - 1 + size) % size;

    int migrationSize = m / 20;
    double* toSend = new double[migrationSize * n];
    double* toRecv = new double[migrationSize * n];
    for (int i = 0; i < migrationSize * n; ++i) {
        toSend[i] = P[i];
    }
    MPI_Sendrecv(toSend, migrationSize * n, MPI_DOUBLE, next, 0,
        toRecv, migrationSize * n, MPI_DOUBLE, prev, 0,
        MPI_COMM_WORLD, &status);
    MPI_Barrier(MPI_COMM_WORLD);
    for (int i = 0; i < migrationSize * n; ++i) {
        P[i] = toRecv[i];
    }
}

```

```

    }

    delete[] toSend;
    delete[] toRecv;
}

void runGA(int n, int m, int t, int rank, int size) {
    int mLocal = m/size;
    double* P = new double[n * mLocal];
    double* pGlobal = nullptr;

    if (rank == 0) {
        pGlobal = new double[n * m];
        init(pGlobal, m, n);
    }
    MPI_Scatter(pGlobal, n*mLocal, MPI_DOUBLE, P, n*mLocal, MPI_DOUBLE, 0,
MPI_COMM_WORLD);

    if (rank == 0) {
        delete[] pGlobal;
    }
    MPI_Barrier(MPI_COMM_WORLD);

    double best = 0;
    double sum = 0;
    double sumGlobal;
    double bestGlobal;
    for (int i = 0; i < t; i++) {
        select(P, mLocal, n);
        crossover(P, mLocal, n);
        mutate(P, mLocal, n);
        best = printthebest(P, mLocal, n);
        sum = best;
        MPI_Allreduce(&sum, &sumGlobal, 1, MPI_DOUBLE, MPI_SUM, MPI_COMM_WORLD);
        MPI_Allreduce(&best, &bestGlobal, 1, MPI_DOUBLE, MPI_MIN,
MPI_COMM_WORLD);
        if (rank == 0) {
            cout << i << " " << bestGlobal << " " << sumGlobal / size << endl;
        }
        if (i % migrationInterval == 0) {
            migrate(P, mLocal, n, rank, size);
            MPI_Barrier(MPI_COMM_WORLD);
        }
    }
    delete[] P;
}

int main(int argc, char** argv) {
    int size;
    int rank;

```

```

MPI_Init(&argc, &argv);
MPI_Comm_size(MPI_COMM_WORLD, &size);
MPI_Comm_rank(MPI_COMM_WORLD, &rank);

if (argc != 8) {
    if (rank == 0) {
        cout << "Initial data is needed" << endl;
    }
    MPI_Barrier(MPI_COMM_WORLD);
    MPI_Finalize();
    return 0;
}

int n = atoi(argv[1]);
int m = atoi(argv[2]);
int t = atoi(argv[3]);
mutationP = atoi(argv[4]);
mutationLen = atoi(argv[5]);
migrationInterval = atoi(argv[6]);
evalFunction = atoi(argv[7]);

if (m % size != 0) {
    if (rank == 0) {
        cout << "m должно быть кратно числу процессов" << endl;
    }
    MPI_Barrier(MPI_COMM_WORLD);
    MPI_Finalize();
    return 0;
}

srand(time(NULL)*(rank+1));
runGA(n, m, t, rank, size);

MPI_Barrier(MPI_COMM_WORLD);
MPI_Finalize();

return 0;
}

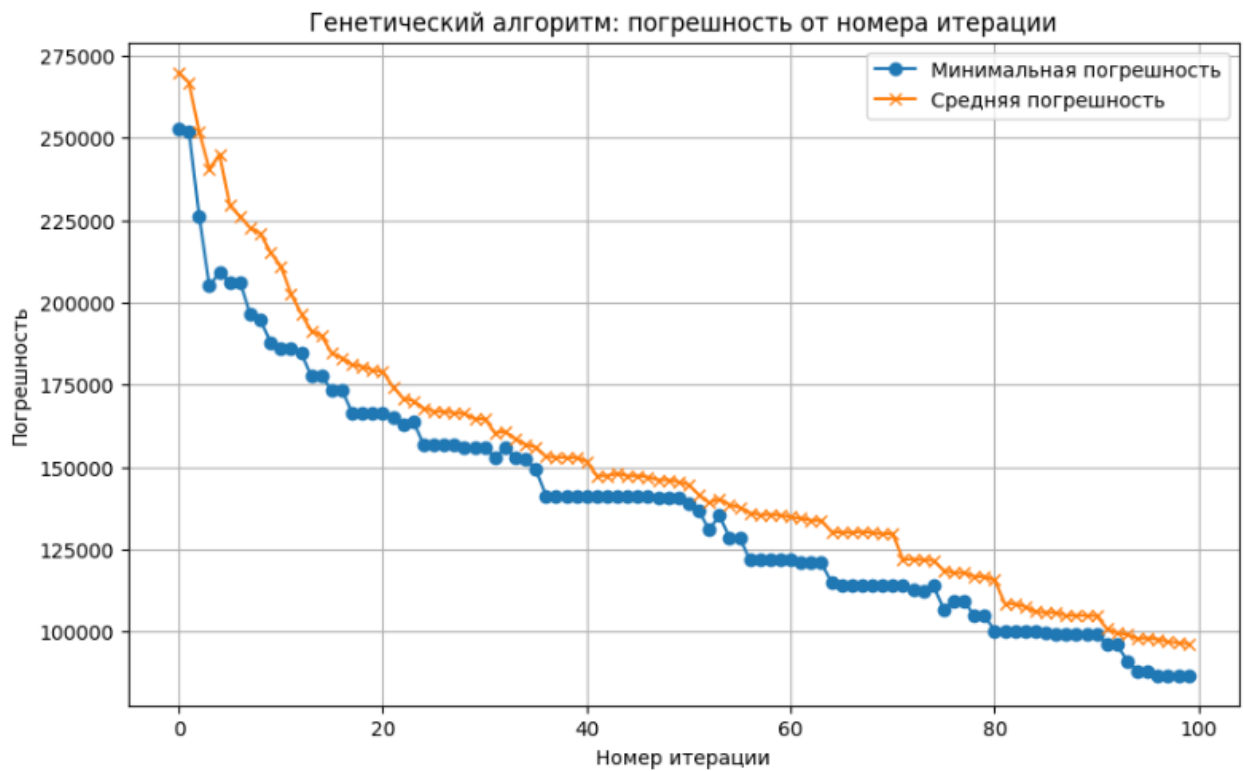
```

2 Графики

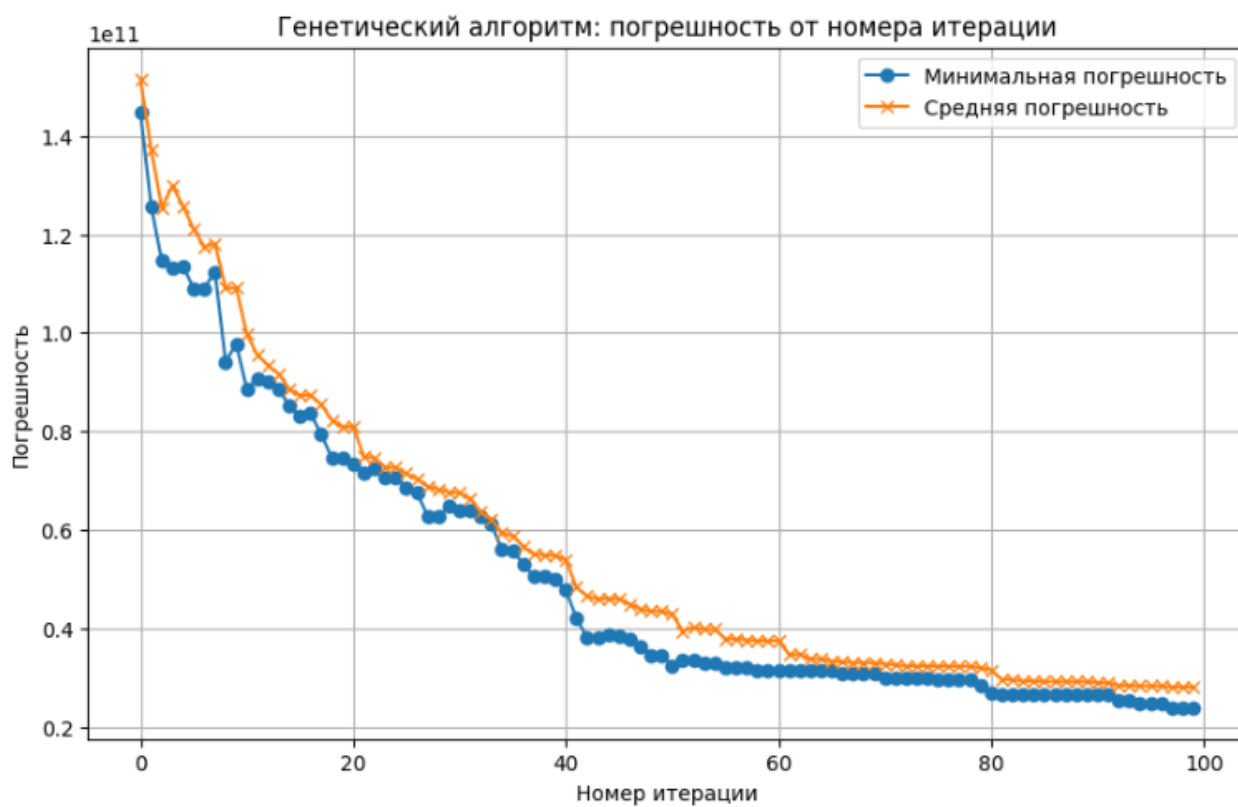
Графики построены с использованием следующих параметров:

- Число генов 100
- Размер популяции 100
- Число итераций 100
- Вероятность мутации 0.1
- Амплитуда изменения гена при мутации 10
- Частота миграций 10

Сферическая функция



Функция Розенброка



Функция Растргина

